A buy that validates before changing state

Studied 17 October 2026; integrated 18 October 2026. Task: L001d, one validated buy. Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can one C++ function either update both cash and shares correctly or reject an order with both fields unchanged? Prediction before implementation: buying two shares at 50 with fee 3 takes cash 800 and four shares to cash 697 and six shares. Cash 102 cannot fund the same order; rejection must preserve cash 102 and four shares, including charging no fee.

L001b established the equations; L001c grouped cash and shares. Today adds one operation: try_buy. A Python driver supplies 14 fixed synthetic calls, compares hand-authored outcomes, runs each build twice and tests two broken variants. There is no historical data, strategy, parameter search or empirical claim. No CLI, shared header or event loop is added.

Beginner explanation and terms

State means the quantities currently held: cash and one asset's shares. An order asks to add a positive number of shares at a specified fill price and a separate fee. Validation checks the request before changing state. Accepted means both fields change; rejected means neither does. The Boolean return is true for acceptance and false for rejection. It does not explain why.

A reference parameter, written AccountState& state, lets the function access the caller's actual account. Without the ampersand the function would receive a copy. This is why an accepted call can change the caller's cash and shares, and why rejection must be tested on that same object. For this lesson, atomic rejection means all-or-nothing at the function boundary in a single thread. It is not a database transaction or a guarantee about concurrent access or crashes.

Model contract and units

Quantity	Admitted values	Meaning
Starting cash and shares	Integers from 0 to 10000	Whole money units and whole shares
Buy quantity and fill price	Integers from 1 to 10000	Shares and money per share
Separate fee	Integer from 0 to 10000	Money units, charged only if accepted
Final shares	At most 10000	Explicit educational position cap

The cap is a chosen teaching constraint, not a broker rule or a machine maximum. Invalid starting state is rejected unchanged; it is not silently repaired. Zero quantity is rejected, while a zero fee is allowed. Cash cannot become negative. No borrowing, shorting or fractional unit is supported. The caller must already supply integers; implicit conversion of a fractional argument before the call is not caught here. Remember the years-parser lesson: text parsing is separate.

Equations and hand calculation

D = n*p + f; C_new = C-D; q_new = q+n. D is the debit, n bought shares, p the fill price, f the fee, C cash and q existing shares. Affordability requires D <= C. At the unchanged mark p, W_new = C_new + q_new*p = C + q*p - f. A fee reduces wealth even though buying exchanges cash for an asset. With C=800, q=4, n=2, p=50 and f=3: D=103, C_new=697, q_new=6 and W_new=997.

Read the function in order

The source is cpp/projects/portfolio_accounting/validated_buy.cpp. Its first declaration repeats the previous two-int AccountState struct. This self-contained lesson does not yet introduce a shared header. The next declaration is bool try_buy(AccountState& state, int quantity, int price_units, int fee_units). bool is the result type; the parameters name the inputs.

The local constant limit is 10000. The first if rejects negative or out-of-domain state, nonpositive quantity or price, and negative or oversized fees. The operator || means that any one failed condition triggers rejection. return false exits immediately. No assignments to state occur in this branch. Checking price before division prevents division by zero.

The next if rejects fee_units > state.cash_units. Now subtracting the fee is safe and nonnegative: available_units = state.cash_units - fee_units. This is a local calculation; it has not deducted a fee from the account. No cash is charged for a rejected request.

The next if compares quantity with available_units / price_units. For nonnegative integers this quotient counts how many complete shares fit after reserving the fee. It also compares quantity with limit - state.shares, the remaining position capacity. Either failure exits before mutation. Equality is allowed: an order may use the last cash unit or reach the cap.

Only after all guards does debit_units calculate quantity * price_units + fee_units. state.cash_units -= debit_units subtracts the debit from the actual account; state.shares += quantity adds the bought shares. Finally return true reports success. There is no later rejecting branch, allocation or throwing operation between these two integer assignments.

Why the order of checks matters

For C=102, fee=3 and price=50, available cash is 99. Integer division gives 99/50 = 1, so quantity 2 is rejected with state 102/4 intact. For C=103 the quotient is 100/50 = 2: the buy is accepted and ends with cash 0 and six shares. Checking only the gross cost of 100 would incorrectly accept the first order or silently omit the fee.

Before multiplication, the affordability guard proves n*p <= C-f. Thus product and final debit are at most 10000. The share-cap guard proves q+n <= 10000. All subtraction operands are bounded and nonnegative at the points where they are evaluated. These intermediates fit even the minimum required positive range of int. A request for 10000 shares at price 10000 is rejected using division; the function never evaluates its large product.

Merely bounding each input would not establish safe multiplication. The guard order is part of the argument. Do not move the product before the affordability check or subtract fees from state during validation. A larger domain needs a fresh arithmetic argument and tests. This function does not calculate marked wealth for arbitrary large values; valuation is separate.

Costs and failure scenarios

At unchanged fill/mark, the accepted example loses exactly fee 3. Spread, slippage, taxes, financing, partial fills, settlement and market impact are absent. The price is assumed known and executable. A rejected order costs nothing under our model; real arrangements can differ. One prevalidated full fill is not a trading strategy or a realistic order-routing simulator. A Boolean result is intentionally simple but gives no diagnostic rejection reason.

Checks, observations and reproduction

Run from the repository root: `.venv/bin/python scripts/verify_validated_buy.py`. The verifier constructs a temporary driver, freshly compiles C++17 with warnings as errors and the undefined-behavior sanitizer, runs twice, checks exact stdout/empty stderr/exit 0, and removes its temporary build. It requires a compiler with the stated sanitizer support. An actual Python -O invocation is refused, preventing disabled assertions from claiming success.

Check group	Observed result	What it establishes
Normal, exact cash and zero fee	3 accepted	Correct final cash and shares for fixed examples
Invalid, unaffordable or capped	11 rejected unchanged	Both fields preserved in these cases
Omitted-fee scratch mutation	Verifier fails as intended	Wrong accepted debit is observable
Fee deducted on rejection	Verifier fails as intended	Partial rejected-state mutation is observable

Hand-authored expected tuples cover zero/negative quantity, zero price, negative fee, invalid cash/shares, fee affordability, domain/position caps and a rejected large product. For accepted cases, a Python per-share debit also checks the hand result and marked-wealth identity. Two complete verifier invocations produce identical JSON, with no sanitizer diagnostics. The first correct implementation passed. Formatting was applied before execution; no failures were hidden. The two wrong variants compile but fail exact-output checks with verifier exit 1.

Full project make check passes 608 Python tests, Ruff, formatting, strict package typing, 66 savings and 30 parser checks, 75 parser-oracle comparisons, existing output fixtures, fractional controls, readiness traces and the prior accounting checks. The integrated struct check is now included. The integrated verifier is now part of make cpp-validated-buy and make check. pip check passes with the existing disabled-cache warning. No dependency or coverage percentage is added.

The current 14 examples are implementation checks, not exhaustive domain verification. L001e will independently generate boundary cases, including negative price and each cap edge, using a broader Python reference. A sanitizer tests executed paths; it does not prove all inputs safe. The algebra above complements the observed cases. Neither proves timing or market realism.

Interview explanation

I validate state, quantity, price, fee and affordability before mutating the caller's account. A division check establishes affordability before multiplying, and a position-cap check bounds the share addition. Accepted calls debit price times quantity plus the fee. Rejected calls leave both fields unchanged. I verify the return flag and each field, including a mutation that charges a fee on rejection. The result is a bounded educational operation, not a backtester.

Progress and publication

October 17's sole commit integrated the earlier struct lesson; L001d was saved pending. On October 18, all 11 input and eight artifact hashes matched before integration. The one available commit publishes L001d with its true study date, original results and reverified PDF. No October 16 lesson artifact was found; no completion or personal review is inferred.

Next learning is L001e independent boundaries, then L001f reproduction. L002 decision/execution timing and future-price invariance remain unfinished; October 19 trend work remains conditional. C003b/C004/C005 and Q004a/Q005b data gates remain. No input blocks synthetic continuation. Continue at 08:00 Europe/London through December 31, then request the next programme scope.

Primary sources consulted on October 17

Microsoft, Functions (C++): function parameters and return types. Our yes/no buy contract and financial assumptions are project choices, not claims made by the language documentation. https://learn.microsoft.com/en-us/cpp/cpp/functions-cpp?view=msvc-170

Microsoft, References (C++): reference access to the same object. We use only a named mutable lvalue reference; no pointers or rvalue-reference lesson is required here. https://learn.microsoft.com/en-us/cpp/cpp/references-cpp?view=msvc-170

Microsoft, Multiplicative Operators: integer division, conversion and division by zero. The affordability-before-multiplication argument is derived here for our nonnegative bounded domain. https://learn.microsoft.com/en-us/cpp/cpp/multiplicative-operators-and-the-modulus-operator?view=msvc-170

Clang, UndefinedBehaviorSanitizer: compile/link flags and runtime diagnostics for executed code. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Python, subprocess: captured streams, return status and bounded waiting for child processes. https://docs.python.org/3/library/subprocess.html#subprocess.run

Check your understanding

1. Why is AccountState& different from AccountState as a parameter?

2. Why must cash 102 reject a two-share buy at 50 with fee 3? What happens at cash 103?

3. Why calculate available cash in a local variable rather than deducting the fee immediately?

4. Why does the division guard come before quantity times price?

5. What does unchanged-state rejection fail to establish about a real trading system?

Suggested answers

1. The reference accesses the caller's object, so accepted updates persist. A value parameter would copy it; editing that copy alone would not change the caller's holdings.

2. The debit is 2*50+3=103. Cash 102 is insufficient, so both original fields remain. Cash 103 permits the buy, leaving zero cash and two additional shares.

3. A local reserve calculation checks affordability without charging a rejected order. Early mutation could leave the account poorer after the function returns false.

4. It proves the product fits the available cash before evaluating it. Individually bounded quantity and price do not by themselves guarantee their product fits int. Positive price is checked first so division is defined; the position guard separately bounds share addition.

5. It says nothing about concurrent threads, crashes, external broker fills, settlement, execution latency, information timing or profitability. Those need separate models and evidence.